

```
In [ ]: # Execute each and every package and methods

# 1. random
# randint
# random
# choice
# sample
# seed
```

```
In [5]: import random

value = random.randint(1,100)
print("Random value:",value)
```

Random value: 9

```
In [9]: import random
help(random.random)
value = random.random()
print("random value:",value)
```

Help on built-in function random:

random() method of random.Random instance
random() -> x in the interval [0, 1).

random value: 0.4268981194088032

```
In [24]: import random
help(random.choice)
l = [12 , 23 ,45 , 34.6 , "pankaj"]
value = random.choice(l)
print("random Choice",value)
```

Help on method choice in module random:

choice(seq) method of random.Random instance
Choose a random element from a non-empty sequence.

random Choice pankaj

```
In [28]: import random
#help(random.sample)
l = [12 , 23 ,45 , 34.6 , "pankaj"]
value = random.sample(l,2)
print("Select two list :",value)
```

Select two list : [12, 45]

```
In [35]: # seed
import random
help(random.seed)
value= random.seed(5)

print(random.randint(1,100))
```

Help on method seed in module random:

seed(a=None, version=2) method of random.Random instance
Initialize internal state from a seed.

The only supported seed types are None, int, float, str, bytes, and bytearray.

None or no argument seeds from current time or from an operating system specific randomness source if available.

If *a* is an int, all bits are used.

For version 2 (the default), all of the bits are used if *a* is a str, bytes, or bytearray. For version 1 (provided for reproducing random sequences from older versions of Python), the algorithm for str and bytes generates a narrower range of seeds.

80

```
In [ ]: # 2.Math
        # sqrt
        # pow
        # pi
        # e
```

```
In [43]: import math
        help(math.sqrt)
        value = math.sqrt(25)
        print("Square", value)
```

Help on built-in function sqrt in module math:

sqrt(x, /)
Return the square root of x.

Square 5.0

```
In [44]: import math
        value = math.pow(2,3)
        print("Power :",value)
```

Power : 8.0

```
In [45]: import math
        print(math.pi)
```

3.141592653589793

```
In [48]: import math
        math.e
```

Out[48]: 2.718281828459045

```
In [49]: import time
        time.sleep(3)
        print("hello")
```

hello

```
In [55]: import time
         help(time.time)
         print(time.time())
```

Help on built-in function time in module time:

```
time()
    time() -> floating-point number
```

Return the current time in seconds since the Epoch.
Fractions of a second may be present if the system clock provides them.

1786614696.6853495

```
In [58]: #4.keywords
         #kwlist

         import keyword
         print(keyword.kwlist)
```

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

```
In [ ]: # 5.os
         # getcwd
         # makedirs
```

```
In [60]: import os
         help(os.getcwd)
         print(os.getcwd())
```

Help on built-in function getcwd in module nt:

```
getcwd()
    Return a unicode string representing the current working directory.
```

C:\Users\panka\Documents\Nareshit\Assignment Gen Ai

```
In [67]: import os
         help(os.makedirs)
         value=os.makedirs("folder")
         print(value)
```

Help on function makedirs in module os:

```
makedirs(name, mode=511, exist_ok=False)
    makedirs(name [, mode=0o777][, exist_ok=False])
```

Super-mkdir; create a leaf directory and all intermediate ones. Works like mkdir, except that any intermediate path segment (not just the rightmost) will be created if it does not exist. If the target directory already exists, raise an OSError if exist_ok is False. Otherwise no exception is raised. This is recursive.

None

```
In [72]: # 6.Date
         # today date
```

```
from datetime import date
print(date.today())
```

2026-08-13

```
In [ ]: # 7. numpy
#       min
#       max
#       sum
```

```
In [74]: import numpy
l = [23, 34 , 45 , 67 ,78]
print(min(l))
```

23

```
In [75]: import numpy
l = [23, 34 , 45 , 67 ,78]
print(max(l))
```

78

```
In [82]: import numpy
l = [23, 34 , 45 , 67 ,78]
print(numpy.sqrt(l))
```

[4.79583152 5.83095189 6.70820393 8.18535277 8.83176087]